



SweetOrder

Sistema de Pedidos de Confeitaria

Guia Completo de Implementação

Passo a Passo para o Desenvolvedor

FastAPI

RabbitMQ

Python

Docker

JWT

SQLAlchemy

Índice de Conteúdo

01	Visão Geral do Projeto	3
02	Pré-requisitos de Instalação	4
03	Estrutura do Repositório	5
04	Configuração do Ambiente	6
05	Subindo o RabbitMQ com Docker	7
06	Desenvolvendo a API (FastAPI)	8
07	Desenvolvendo o Worker	10
08	Desenvolvendo o Frontend	11
09	Autenticação JWT	12
10	CI/CD com GitHub Actions	13
11	Testando o Sistema Completo	14
12	Checklist Final de Entrega	15

01 — Visão Geral do Projeto

O **SweetOrder** é um sistema de e-commerce de confeitaria que demonstra integração entre múltiplos componentes usando arquitetura assíncrona. O objetivo acadêmico é mostrar como um Frontend, uma API RESTful, uma fila de mensagens e um Worker se comunicam de forma desacoplada e escalável.

Stack Tecnológica

Tecnologia	Função
Frontend	HTML5 + CSS3 + JavaScript Vanilla — interface do usuário
FastAPI	API Gateway RESTful que recebe pedidos e publica na fila
RabbitMQ	Message broker para comunicação assíncrona entre serviços
Worker Python	Consumidor da fila que processa e aprova/rejeita pedidos
SQLite / PostgreSQL	Banco de dados para persistência dos pedidos
JWT (Bearer Token)	Autenticação e autorização das rotas da API
Docker Compose	Orquestração de todos os serviços em containers
GitHub Actions	Pipeline de CI/CD com testes automáticos

Fluxo de um Pedido

- 1■■ Usuário faz login no Frontend e obtém um **JWT Token**
- 2■■ Usuário preenche o formulário e clica em **Fazer Pedido**
- 3■■ Frontend envia **POST /orders** com o token no header
- 4■■ API salva o pedido no banco com status **PENDENTE**
- 5■■ API publica mensagem na fila **orders_queue** do RabbitMQ
- 6■■ API retorna **201 Created** com o ID do pedido
- 7■■ Worker consome a mensagem e **simula aprovação (5 segundos)**
- 8■■ Worker atualiza o banco: status **APROVADO** ou **REJEITADO**
- 9■■ Frontend faz polling a cada 2s em **GET /orders/{id}** e exibe resultado

1

Pré-requisitos de Instalação

Instale as ferramentas antes de começar

Ferramentas Necessárias

- Git — controle de versão (git --version)
- Python 3.11+ — linguagem backend (python --version)
- Docker Desktop — para rodar RabbitMQ e API em containers
- VS Code — editor recomendado (com extensão Live Server)
- Extensão REST Client ou Insomnia — para testar a API
- Conta no GitHub — para o repositório e CI/CD

Comandos de Verificação

```
git --version # deve retornar git version 2.x
```

```
python --version # deve retornar Python 3.11.x
```

```
docker --version # deve retornar Docker version 24.x
```

```
docker compose version # deve retornar Docker Compose version 2.x
```

■ **Dica:** No Windows, certifique-se de que o Docker Desktop está em execução antes de qualquer comando docker.

2

Criando a Estrutura do Repositório

Organize as pastas antes de escrever qualquer código

Criando o Projeto no GitHub

1. Acesse github.com e crie um novo repositório chamado **sweetorder**
2. Marque a opção **Add a README file**
3. Clone o repositório localmente:

```
git clone https://github.com/SEU_USUARIO/sweetorder.git cd sweetorder
```

Criando a Estrutura de Pastas

```
mkdir -p frontend api/routers api/models api/schemas api/services api/tests mkdir -p worker .github/workflows
```

Estrutura Final Esperada

```
sweetorder/ ■■■ frontend/ → HTML, CSS e JavaScript ■■■ api/ → FastAPI (API Gateway) ■  
■■■ routers/ → Endpoints: auth.py, orders.py ■ ■■■ models/ → ORM SQLAlchemy ■ ■■■  
schemas/ → Pydantic schemas ■ ■■■ services/ → JWT e RabbitMQ helpers ■ ■■■ tests/ →  
Pytest ■■■ worker/ → Consumer RabbitMQ ■■■ .github/workflows/ → GitHub Actions CI ■■■  
docker-compose.yml ■■■ .env.example
```

3

Configuração do Ambiente

Crie o arquivo `.env` e configure as variáveis

Criando o `.env.example`

```
# Cole isso em .env.example na raiz do projeto
SECRET_KEY=troque_por_uma_chave_segura_de_32_chars ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30 RABBITMQ_HOST=rabbitmq RABBITMQ_PORT=5672
RABBITMQ_USER=sweet_user RABBITMQ_PASS=sweet_pass RABBITMQ_QUEUE=orders_queue
DATABASE_URL=sqlite:///./sweetorder.db API_PORT=8000
```

```
cp .env.example .env # crie o arquivo real
```

■ ■ **Atenção:** Adicione `.env` no `.gitignore` imediatamente! Nunca commite credenciais reais.

Ambiente Virtual Python

```
cd api python -m venv venv source venv/bin/activate # Linux/Mac # ou: venv\Scripts\activate
# Windows pip install -r requirements.txt
```

4

Subindo o RabbitMQ com Docker

Configure o message broker antes da API

Criando o docker-compose.yml

```
version: '3.8' services: rabbitmq: image: rabbitmq:3-management ports: - '5672:5672' #
conexão da aplicação - '15672:15672' # painel web de administração environment:
RABBITMQ_DEFAULT_USER: sweet_user RABBITMQ_DEFAULT_PASS: sweet_pass healthcheck: test:
['CMD', 'rabbitmq-diagnostics', 'ping'] interval: 10s timeout: 5s retries: 5 api: build:
./api ports: - '8000:8000' env_file: .env depends_on: rabbitmq: condition: service_healthy
worker: build: ./worker env_file: .env depends_on: rabbitmq: condition: service_healthy
```

Iniciando os Containers

```
docker compose up --build -d # Verificar se está rodando: docker compose ps # Ver logs da
API: docker compose logs api -f # Ver logs do worker: docker compose logs worker -f
```

■ **Dica:** Acesse o painel do RabbitMQ em <http://localhost:15672> (sweet_user / sweet_pass) para ver as filas.

5

Desenvolvendo a API (FastAPI)

Implemente os endpoints RESTful com autenticação

Passe o arquivo AGENTS.md para o Cursor/IA e peça:

Com o arquivo **AGENTS.md** aberto no Cursor, use o chat e digite:

```
"Implemente o projeto SweetOrder conforme o AGENTS.md. Comece pelo arquivo api/main.py e depois os routers."
```

Endpoints que devem existir

Tecnologia	Função
POST /auth/token	Login — retorna JWT Bearer Token
POST /orders	Criar pedido (requer auth) — publica na fila
GET /orders/{id}	Consultar status de um pedido (requer auth)
GET /orders	Listar todos os pedidos com filtros (requer auth)
GET /health	Health check — verifica se a API está viva
GET /docs	Swagger UI automático do FastAPI

Testando a API manualmente

```
# 1. Obter token curl -X POST http://localhost:8000/auth/token \ -d
'username=admin&password=confeitaria2025' # 2. Criar pedido (substitua SEU_TOKEN) curl -X
POST http://localhost:8000/orders \ -H 'Authorization: Bearer SEU_TOKEN' \ -H 'Content-Type:
application/json' \ -d
'{"customer_name":"Maria","items":[{"product":"Bolo","quantity":1,"unit_price":89.90}]}' #
3. Consultar status curl http://localhost:8000/orders/ID_DO_PEDIDO \ -H 'Authorization:
Bearer SEU_TOKEN'
```


6

Desenvolvendo o Worker

Crie o consumidor assíncrono da fila

Lógica do Worker (worker/worker.py)

```
import pika, json, time, random, logging from datetime import datetime # ... imports
SQLAlchemy ... def process_order(ch, method, properties, body): data = json.loads(body)
order_id = data['order_id'] logging.info(f'Processando pedido {order_id}...') # Simula
aprovação de pagamento time.sleep(5) # 80% de chance de aprovação status = 'APROVADO' if
random.random() < 0.8 else 'REJEITADO' # Atualiza banco de dados
update_order_status(order_id, status) logging.info(f'Pedido {order_id}: {status}')
ch.basic_ack(delivery_tag=method.delivery_tag) # Conectar ao RabbitMQ e consumir
channel.basic_consume(queue='orders_queue', on_message_callback=process_order)
channel.start_consuming()
```

■ **Dica:** O worker deve rodar em loop infinito. O Docker Compose garante que ele reinicia em caso de falha (restart: unless-stopped).

7

Desenvolvendo o Frontend

Crie a interface visual de confeitaria

Funcionalidades obrigatórias no HTML/JS

- Tela de login com campos username e password
- Chamar POST /auth/token e salvar o token no localStorage
- Formulário de pedido: nome do cliente + lista de itens
- Botão para adicionar mais itens dinamicamente
- Calcular total em tempo real (quantity × unit_price)
- Chamar POST /orders com o Bearer Token no header
- Exibir mensagem: '■ Pedido enviado! Aguardando processamento...'
- Polling a cada 2 segundos em GET /orders/{id}
- Exibir '■ Pedido APROVADO!' ou '■ Pedido REJEITADO'
- Parar o polling quando o status for diferente de PENDENTE

Paleta de Cores (use no CSS)

```
:root { --primary: #C0392B; /* vermelho confeitaria */ --secondary: #F9E4B7; /* bege creme */  
--accent: #8B4513; /* marrom chocolate */ --background: #FFF8F0; /* branco creme */  
--text: #2C1810; /* marrom escuro */ }
```

■ **Dica:** Use o Live Server do VS Code (botão 'Go Live') para abrir o frontend em <http://localhost:5500>

8

Autenticação JWT

Proteja as rotas com Bearer Token

Fluxo de Autenticação

1. **Usuário faz login** → POST /auth/token com username e password
2. **API valida** as credenciais no banco (senha com bcrypt hash)
3. **API gera JWT** com payload: {'sub': username, 'exp': timestamp}
4. **Frontend armazena** o token no localStorage
5. **Frontend inclui** em toda request: Authorization: Bearer {token}
6. **FastAPI valida** via Depends(get_current_user) automaticamente

Usuário de seed (criar na startup da API)

```
# Esse usuário deve ser criado automaticamente # quando a API inicializa pela primeira vez
username: admin password: confeitaria2025 # armazene o hash bcrypt, nunca a senha real
```

■■ **Atenção:** Use `passlib[bcrypt]` para hashear a senha. **NUNCA** armazene senhas em texto puro no banco.

9

CI/CD com GitHub Actions

Automatize testes a cada push

Criando o arquivo de workflow

Crie: .github/workflows/ci.yml

```
name: SweetOrder CI on: push: branches: [main] pull_request: branches: [main] jobs: test:
  runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-python@v5
  with: python-version: '3.11' - name: Instalar dependências run: | cd api pip install -r
  requirements.txt - name: Rodar testes run: | cd api pytest tests/ -v --cov=.
  --cov-report=xml - name: Lint com flake8 run: | pip install flake8 flake8 api/
  --max-line-length=100 --exclude=venv
```

■ **Dica:** Após o primeiro push, acesse a aba 'Actions' no GitHub para ver o pipeline executando automaticamente.

10

Rodando os Testes

Valide o sistema antes de entregar

Testes Automatizados (pytest)

```
cd api source venv/bin/activate pytest tests/ -v # Resultado esperado: # PASSED
tests/test_auth.py::test_login_success # PASSED tests/test_auth.py::test_login_invalid #
PASSED tests/test_orders.py::test_create_order # PASSED tests/test_orders.py::test_get_order
# PASSED tests/test_orders.py::test_unauthorized # ===== 5 passed in 1.23s
=====
```

Teste Manual Completo (fluxo real)

- Docker Compose está rodando (docker compose ps)
- Abrir <http://localhost:15672> e ver a fila 'orders_queue'
- Abrir o frontend no Live Server (<http://localhost:5500>)
- Fazer login com admin / confeitaria2025
- Criar um pedido com ao menos 2 itens
- Observar status 'PENDENTE' aparecer na tela
- Aguardar ~5-8 segundos e ver o status mudar para APROVADO ou REJEITADO
- Verificar no Swagger (<http://localhost:8000/docs>) que o pedido existe
- Verificar nos logs do worker: docker compose logs worker

11

Checklist Final de Entrega

Verifique tudo antes de apresentar

Repositório GitHub

- Repositório público no GitHub com nome 'sweetorder'
- README.md com instruções de instalação e uso
- Pastas organizadas: frontend/, api/, worker/
- .env.example commitado (nunca o .env real)
- .gitignore incluindo: .env, __pycache__, venv/, *.db
- GitHub Actions badge no README mostrando CI passando

Entregáveis Técnicos

- Diagrama de arquitetura (Miro ou Draw.io)
- Swagger UI rodando em `http://localhost:8000/docs`
- Print ou export do Swagger em PDF
- Testes pytest com cobertura mínima de 70%
- Pipeline CI/CD funcionando no GitHub Actions

Apresentação (Pitch de 10 minutos)

- Slide 1: Problema e solução (confeitaria precisando de pedidos online)
- Slide 2: Arquitetura — diagrama explicando cada componente
- Slide 3: Demo ao vivo — criar pedido e ver aprovação em tempo real
- Slide 4: Código — mostrar um trecho da API e do Worker
- Slide 5: CI/CD — mostrar o GitHub Actions passando
- Slide 6: Desafios encontrados e como foram resolvidos

-
- **Boa sorte na implementação!** Siga o AGENTS.md com o Cursor e este guia passo a passo. Em caso de dúvidas, verifique os logs com `docker compose logs [serviço] -f`